CASS: Training-Free Adaptation to Unseen Tasks by Mining and Composing Skill Subspaces from LLM Activations

Anonymous Author(s)

Abstract

Large language models encode task information as manipulable structure in their activation space: task vectors extracted from in-context demonstrations causally trigger task execution when injected elsewhere. Existing steering approaches, however, treat each task in isolation—vectors are extracted per task, stored, and retrieved, leaving models unable to exploit shared structure when a novel task arrives. We ask: what part of a new task's steering operator can be *composed* from a dictionary of previously mined skills, and how should the composable and task-specific parts be combined? We present CASS (Compositional Adaptive Subspace Steering), a fully training-free framework that (i) mines a dictionary of low-rank skill subspaces from contrastive activations at two residual-stream depths, after projecting out a shared task-generic component that we identify as the dominant cause of interference in naive vector composition; (ii) given $k \leq 4$ demonstrations of a novel task, solves a joint group-sparse coding problem—with closed-form block updates and support shared across layers—to identify the relevant skills; and (iii) steers the model with a query-adaptive affine operator that combines the demonstrations' denoised activation direction with a subspace correction derived from the recovered support. We prove support-recovery guarantees under a block-coherence condition and bound steering sub-optimality by the coding residual, which doubles as a practical reliability signal. On 32 tasks and Llama-3.1-8B, CASS recovers a median 86% of oracle task-vector performance on held-out tasks from only 4 examples (naive composition: 0%), identifies the constituent skills of compound tasks with 0.68 recall, improves monotonically with dictionary size, and amortizes to a 10× per-query token saving over full in-context prompting beyond ∼17 queries per task.

CCS Concepts

• **Computing methodologies** → **Machine learning**.

Keywords

large language models, activation steering, task vectors, sparse coding, representation mining

ACM Reference Format:

Anonymous Author(s). 2027. CASS: Training-Free Adaptation to Unseen Tasks by Mining and Composing Skill Subspaces from LLM Activations. In *Proceedings of The 33rd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '27).* ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

KDD '27, TBD

2027. ACM ISBN 978-x-xxxx-xxxx-x/27/08

https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large-scale LLM serving pipelines face a long tail of lightweight, recurring tasks. The standard remedy—prepending k-shot demonstrations to every query—multiplies prompt length by an order of magnitude and is paid again on every call: at 10-shot prompting, a task served a thousand times costs ∼131k prompt tokens where the queries alone would cost ∼10k (Table **??**). A growing body of work shows that the demonstrations' effect is substantially captured by compact structure in the model's activation space: *task vectors* or *function vectors* distilled from in-context prompts can be injected into the residual stream to trigger task execution without any demonstrations in context [5, 8]. Activation-level task structure is not an artifact of one probe: different tasks induce partitioned, clusterable hidden representations across layers [11].

Yet existing steering methods treat tasks as islands. Per-task vectors are extracted offline and retrieved by similarity at serving time [10]; adaptive variants train a generator network to map demonstrations to vectors [1]. Neither can serve a task absent from the library without new training, and single-vector representations provably degrade on tasks whose activation signature is high-rank [2]. The blind spot is *unseen* tasks: can a steering operator for a task never seen before be assembled, at negligible cost, from skills mined earlier?

Naive assembly fails. Simply averaging dictionary vectors steers the model to 0% of oracle performance in our experiments (§6.1). We trace this failure to a structural property: every contrastively extracted task vector shares a large *task-generic* component—the signature of "an ICL task is being executed"—whose superposition drowns task-specific directions. Projecting out a rank-1 shared subspace drops the median pairwise cosine between task means from 0.34 to 0.18 and doubles downstream composition accuracy (§6.3). Once removed, a second structure appears: the residual task vectors are *partially* compositional. A held-out task's representation is well approximated inside the span of a handful of related skills—its group-sparse code selects semantically correct neighbors (currency→{capital, landmark, park}; French→{German, Spanish})—while an irreducible task-specific component remains outside every other task's subspace. Effective adaptation must therefore *combine* the composable part (which the dictionary can identify, denoise, and correct toward) with the task-specific part (which only the k demonstrations carry).

CASS operationalizes this in three training-free stages (Fig. **??**): dictionary mining with common-component removal (§4.1); joint group-sparse coding across two injection layers with closed-form block updates, millisecond solve times, and a shared support (§4.2); and a query-adaptive affine steering operator whose direction is the demonstrations' denoised contrastive activation and whose correction pulls hidden states toward the recovered skill subspace (§4.3). The coding residual ε bounds steering sub-optimality (Prop. 2)

and doubles as a deployment-time reliability signal: when ε is large the system can fall back to full ICL.

Contributions. (1) The first training-free framework that adapts activation steering to unseen tasks by composing mined skill subspaces, with an explicit, interpretable decomposition into composable and task-specific parts. (2) A mechanism finding: a shared task-generic activation component is the dominant cause of interference in vector composition; removing it (at the right rank) is necessary for compositionality to emerge, doubling composition accuracy, with a sharply non-monotone rank ablation. (3) Theory: support-recovery guarantees for the joint multi-layer group-sparse code under a block-coherence condition, and an error-transfer bound linking coding residual to steering performance—together with measured coherence values that delineate where the theory's conditions hold (within-family coherence violates them exactly where we observe wrong-neighbor steering). (4) A scaling and cost narrative for activation mining as a reusable asset: composition quality grows monotonically with dictionary size, dictionaries need only 25 contrastive samples per skill, and serving amortizes to a 10× token saving over ICL at 10^3 queries.

2 Related Work

Task and function vectors. Hendel et al. [5] and Todd et al. [8] show ICL compresses task identity into injectable vectors; follow-ups analyze their geometry and limits, including rank limitations of single vectors [2]. These works extract and analyze *per-task* representations; we *synthesize* operators for unseen tasks and use subspaces as primitives, directly addressing the high-rank failure.

Vector libraries and adaptivity. ELICIT [10] retrieves from a capability library; ATV [1] trains a small network to generate task vectors. Retrieval cannot leave the library of known tasks, and generation requires training. CASS is training-free and returns an explicit compositional explanation (the sparse code) with each operator.

Activation steering. Steering vectors add fixed directions for known behaviors [9, 13]; conceptors generalize to ellipsoidal regions and Boolean combinations of *known* behaviors [7]. We instead *infer* the combination for an unknown task from k examples; conceptor-style composition enters our experiments as the anchor-blend ablation.

Mining task structure in activations. SEAP [11] clusters task activations to prune computation; contextual-sparsity methods exploit similar structure for efficiency [6]. We use the same raw material—activation data across tasks—but mine it into a reusable skill dictionary that *adds* capability (steering to unseen tasks) rather than removing computation.

3 Skill Structure in Activation Space

Setup and notation. Let M be a decoder-only LM with L layers and residual width d (L=32, d=4096 for Llama-3.1-8B). For prompt p, $h_\ell(p) \in \mathbb{R}^d$ is the residual-stream state of the last token at layer ℓ. We assume a base task set $\mathcal{T} = \{1, \dots, T\}$ with datasets of input–output pairs, organized in five families (knowledge mapping, linguistic transforms, translation, algorithmic/symbolic, list selection); T=32 in our experiments.

Contrastive extraction. For task t and sample i we build a 10-shot ICL prompt $p_i^{t,+}$ and a corrupted twin $p_i^{t,-}$ with the same inputs but derangement-shuffled labels (format and vocabulary preserved, task mapping destroyed), and record the differential activation $g_i^{(t)} = h_\ell(p_i^{t,+}) - h_\ell(p_i^{t,-})$ at every layer in one forward pass. Per task we use n=100 pairs (n=25 suffices; §6.3).

The shared task-generic component (H1). Stacking task means $\bar{g}_t$ reveals a dominant shared direction: the top left-singular vector of $[\bar{g}_1 \cdots \bar{g}_T]$ captures format-following and answer-emission signals common to all ICL tasks. It is the primary source of interference when vectors are added: with it, the median pairwise $|\cos(\bar{g}_s, \bar{g}_t)|$ is 0.34; projecting out rank r_0=1 lowers this to 0.18, and composition accuracy doubles (0.25 → 0.50, Table **??**). Removing too much ($r_0 \geq 8$) destroys the very directions steering needs—the curve is sharply non-monotone, which supports reading U_0 as a real, low-rank shared component rather than generic denoising.

4 CASS

4.1 Module 1: Mining the Skill Dictionary

Fix a small set of injection layers $\mathcal{L}$ ($|\mathcal{L}|$=2; selection below). Per layer ℓ: compute the shared component $U_0^{(\ell)} \in \mathbb{R}^{d \times r_0}$ by SVD of stacked task means; project it out of all differential activations, $\tilde{G}_t^{(\ell)} = (I - U_0^{(\ell)} U_0^{(\ell)\top}) G_t^{(\ell)}$; and take the truncated SVD basis $U_t^{(\ell)} \in \mathbb{R}^{d \times r_{t\ell}}$ covering 90% spectral energy (capped at rank 16), with anchor $\mu_t^{(\ell)}$ = mean of $\tilde{G}_t^{(\ell)}$. The dictionary entry for skill t stacks layers block-diagonally:

$$\mathbf{U}_t = \begin{bmatrix} U_t^{(\ell_1)} & 0 \\ 0 & U_t^{(\ell_2)} \end{bmatrix} \in \mathbb{R}^{2d \times r_t}, \qquad \boldsymbol{\mu}_t = \begin{bmatrix} \mu_t^{(\ell_1)} \\ \mu_t^{(\ell_2)} \end{bmatrix},$$

so columns remain orthonormal and the group-sparse solver below applies unchanged, while coefficients stay layer-specific and the *support* is shared across layers.

Layer selection. A one-time causal scan over single layers, followed by a small grid over pairs using *oracle* injection recovery on held-in tasks, selects $\mathcal{L} = \{12, 16\}$ for Llama-3.1-8B. Two depths matter because task families peak at different layers (knowledge ~12, linguistic ~16–18); joint injection lifts mean oracle recovery from 0.44 (best single layer, additive) to 0.76 with no per-task tuning.

4.2 Module 2: Joint Group-Sparse Coding

At test time a novel task $t^\star$ provides $k \leq 4$ examples. For each example j we form m=6 (clean, corrupted) prompt pairs—shots are the other $k-1$ examples in resampled orders, corruption re-drawn—and average the differential activations into a denoised, stacked, deshared representation $z_j \in \mathbb{R}^{2d}$; $z = \frac{1}{k} \sum_j z_j$. Averaging over prompt resamplings is essential: it raises $\cos(z, \boldsymbol{\mu}_{t^\star})$ from ~0.5 to ~0.7–0.8.

We solve the group LASSO

$$\min_{c_1, \dots, c_T} \ \frac{1}{2} \Big\| z - \sum_t \mathbf{U}_t c_t \Big\|_2^2 + \lambda \sum_t \sqrt{r_t}\, \|c_t\|_2 \qquad (1)$$

by block coordinate descent; orthonormal blocks make each update an exact group soft-threshold, and the full solve takes milliseconds on CPU. λ is set without any validation data by walking the path from $\lambda_{\max}$ downward with warm starts and keeping the best-fitting

solution whose support size is at most $s_{\max}$=5. Outputs: support $S = \{t : \|c_t^*\| > 0\}$, reconstruction $\Delta_{\text{rec}} = \sum_{t \in S} \mathbf{U}_t c_t^*$, and residual $\varepsilon = \|z - \Delta_{\text{rec}}\|/\|z\|$.

4.3 Module 3: Query-Adaptive Hybrid Injection

The recovered support is reliably semantic, but Δ_{rec} alone understeers: a held-out task's vector keeps 40–60% of its energy outside all other skills' subspaces, and that out-of-span component is precisely what distinguishes, e.g.„ French from Spanish. CASS therefore injects the *demonstration direction* and uses the dictionary for *structure*: with support weights $w_t = \|c_t^*\|/\sum_{s \in S} \|c_s^*\|$, anchor $\boldsymbol{\mu}_S = \sum_{t \in S} w_t \boldsymbol{\mu}_t$, subspace projector P_S (QR of concatenated support bases, per layer), and z rescaled to the support-anchor norm, the last-token state at each $\ell \in \mathcal{L}$ is updated as

$$h \leftarrow h + \alpha(h)\big[\gamma\, \Delta^{(\ell)} + P_S^{(\ell)}\big(\mu_S^{(\ell)} - h\big)\big], \qquad \alpha(h) = \min\Big(\alpha_{\max},\ \beta\, \|(I - P_S^{(\ell)})(h - \mu_S^{(\ell)})\|/\|h\|\Big), \tag{2}$$

with Δ the (rescaled) z and $(\gamma, \beta, \alpha_{\max}) = (1, 2, 1)$ fixed once on three held-in tasks and frozen for all experiments and models. Intuition: the further h lies from the synthesized task manifold, the harder it is pulled; states already on-manifold are barely touched. Setting $\Delta = \Delta_{\text{rec}}$ instead (pure synthesis) is the key ablation: it isolates how much of a new task is dictionary-expressible (Table **??**).

5 Theoretical Analysis

Define block coherence $\mu_B = \max_{s \neq t} \sigma_{\max}(\mathbf{U}_s^\top \mathbf{U}_t)$, the largest principal-angle cosine between distinct skill subspaces.

Proposition 1 (Support recovery). *Suppose $z = \sum_{t \in S_0} \mathbf{U}_t c_t^0 + e$ with $|S_0| = k_0$ and $\|e\| \le \epsilon$. If $k_0 < \frac{1}{2}(\mu_B^{-1} + 1)$ and λ is chosen appropriately for ϵ, the group-LASSO solution satisfies $\hat{S} \subseteq S_0$ with coefficient error $O(\epsilon)$.*

The proof instantiates block-sparse recovery results for unions of subspaces [3, 4] and group-LASSO consistency [12]; we contribute the *verification*: common-component removal is what brings measured coherence into a workable regime (median pairwise μ_B vs. r_0; Table **??**), and the condition's failure is diagnostic—the largest coherences occur inside the translation family (μ_B up to 0.95), exactly where we observe correct *family* recovery but wrong-member steering (§6.6).

Proposition 2 (Error transfer). *If task performance is locally linear in the injected state, $m(h) = w^\top h + O(\|h\|_{loc}^2)$, then $|m(h + \Delta) - m(h + \Delta^{or})| \le \|w\|\,(\varepsilon\|z\| + \delta_{est})$, where δ_{est} is the finite-sample subspace estimation error controlled by Davis–Kahan at rate $O(\sigma/(\sqrt{n}\,\text{gap}))$.*

Empirically ε anti-correlates with recovery (Fig. **??**), grounding its use as a deployment-time fallback trigger.

6 Experiments

Setup. 32 tasks from the function-vectors benchmark [8] across five families; per task, 50 held-out evaluation queries disjoint from dictionary and demonstration pools; 5 seeds for demonstration sampling. Main model Llama-3.1-8B-Instruct; cross-model validation on Qwen3-4B and Llama-3.2-3B. Metric: strict first-words exact match (case-sensitive for compound tasks). Recovery $\rho = (acc_{\text{syn}} - acc_{\text{zs}})/(acc_{\text{or}} - acc_{\text{zs}})$ against each task's own-subspace oracle injection; tasks with oracle-zs gap < 5pts are excluded from ρ aggregation and reported separately. A single RTX 3090 suffices for the entire study (~40 GPU-hours).

6.1 E1: Unseen-Task Synthesis (Leave-One-Task-Out)

For each held-out task the dictionary (including U_0) is rebuilt from the remaining 31 tasks. With k=4 examples, CASS attains median ρ = **0.86**; 70% of the 27 valid tasks exceed ρ=0.6 (Fig. **??**). Median ρ is 0.37 at k=1 and already 0.89 at k=2. By family (k=4): linguistic 1.01, selection 0.93, knowledge 0.84, algorithmic 0.65, translation 0.43. Baselines: naive anchor averaging ρ = 0.00; raw demonstration vector without the dictionary ("z-only") trails CASS by 3.9pts mean accuracy at k=4; pure reconstruction trails by 16.8pts, quantifying the irreducible task-specific component.

6.2 E2: Compound Tasks and Interpretability

Ten compound tasks chain two dictionary skills (e.g., `singular-plural∘capitali` none is in the dictionary, and scoring is case-sensitive so the second component is only credited when actually executed. The sparse code recovers constituent skills with recall 0.68 (precision 0.36; coefficient heatmap in Fig. **??**). Steering executes the composition where the components' formats are compatible (`singular-plural+capitalize` 0.86, `present-past+capitalize` 0.50) and CASS is the only non-trivial method overall (0.16 vs. retrieval 0.03, naive component averaging 0.05; 10-shot ICL 0.61): retrieval cannot compose, and its failure here is categorical, not marginal.

6.3 E4: Ablations

Table **??** summarizes (8 representative tasks, k=4, 3 seeds). Common-component removal is load-bearing: r_0=0/1/2/8/16 → mean accuracy 0.25/0.50/0.42/0.17/0.02. Injection layers: {12,16} (0.42) > {12,14,16} (0.40) > single 16 (0.28) > single 12 (0.14). Direction source: hybrid 0.42 > z-only 0.41 ≫ reconstruction 0.10 > anchor blend 0.05. Solver: group LASSO ≈ block-OMP > simplex > least squares. Dictionary build cost: n=25 samples per task matches n=100.

6.4 E5: Dictionary Scaling and Serving Cost

Accuracy on six fixed held-out tasks grows monotonically with dictionary size—0.14/0.44/0.51/0.52 at T'=5/10/15/31 (mean over 5 random sub-dictionaries)—supporting the mined-dictionary-as-asset framing: every added skill improves zero-training transfer to unseen tasks (Fig. **??**). Serving cost (Table **??**): a 10-shot ICL prompt averages 131 tokens vs. 10.5 zero-shot; CASS's one-time extraction costs 2,211 tokens, breaking even at ~17 queries and saving 10.2× at 10^3 queries per task.

6.5 E6: Cross-Model Validation

[E6 running: Qwen3-4B and Llama-3.2-3B with depth-proportional layer pairs; numbers to be inserted.]

6.6 Failure Analysis

Stored generations let us separate failure modes. (i) *Format non-compliance*: on `person-occupation`/`person-sport` the steered model emits biographies containing the answer (contains-metric 0.29 vs. exact 0.02; oracle contains 0.38)—steering conveys the mapping but not the answer-only format. (ii) *Coherent-neighbor attraction*: held-out `english-french` recovers the translation family but is pulled toward Spanish/German anchors by the correction term, producing fluent wrong-language answers; this is the μ_B condition of Prop. 1 failing within-family, observable in advance from the dictionary itself. (iii) ε predicts ρ (Fig. **??**), enabling selective fallback to full ICL.

7 Conclusion

CASS shows that the activation footprint of an unseen task decomposes into a part that is compositional over previously mined skills and an irreducible task-specific remainder—and that a training-free system exploiting both recovers most of oracle steering performance from four examples at a fraction of ICL's serving cost. Mining activation structure once and reusing it across the task long tail turns steering from a per-task artifact into a growing knowledge asset.

References

[1] Anonymous. 2025. Adaptive Task Vectors for Large Language Models. *arXiv preprint* (2025). TODO: verify citation details.

[2] Anonymous. 2026. On the Rank Limitations of Task Vectors for In-Context Learning. In *ICLR*. TODO: verify citation details.

[3] Yonina C. Eldar, Patrick Kuppinger, and Helmut Bölcskei. 2010. Block-Sparse Signals: Uncertainty Relations and Efficient Recovery. *IEEE Transactions on Signal Processing* 58, 6 (2010), 3042–3054.

[4] Yonina C. Eldar and Moshe Mishali. 2009. Robust Recovery of Signals from a Structured Union of Subspaces. *IEEE Transactions on Information Theory* 55, 11 (2009), 5302–5316.

[5] Roee Hendel, Mor Geva, and Amir Globerson. 2023. In-Context Learning Creates Task Vectors. In *Findings of EMNLP*. arXiv:2310.15916.

[6] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, et al. 2023. Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time. In *ICML*.

[7] Joris Postmus and Steven Abreu. 2024. From Steering Vectors to Conceptors and Beyond: Compositional Affine Steering Mechanisms for LLMs. In *OpenReview*. TODO: verify venue.

[8] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. 2024. Function Vectors in Large Language Models. In *ICLR*. arXiv:2310.15213.

[9] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. 2023. Steering Language Models with Activation Engineering. *arXiv preprint arXiv:2308.10248* (2023).

[10] Futing Wang, Jianhao Yan, Yue Zhang, and Tao Lin. 2025. ELICIT: LLM Augmentation via External In-Context Capability. In *ICLR*. arXiv:2410.09343.

[11] Xun Xie et al. 2025. SEAP: Training-free Sparse Expert Activation Pruning Unlock the Brainpower of Large Language Models. *arXiv preprint arXiv:2503.07605* (2025).

[12] Ming Yuan and Yi Lin. 2006. Model Selection and Estimation in Regression with Grouped Variables. *Journal of the Royal Statistical Society: Series B* 68, 1 (2006), 49–67.

[13] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, et al. 2023. Representation Engineering: A Top-Down Approach to AI Transparency. *arXiv preprint arXiv:2310.01405* (2023).

A Reproducibility

Code, task splits, frozen hyperparameters, and per-run generations are released; all experiments run on one 24GB GPU. [Proofs, full per-task tables, and additional ablations to be inserted.]